

Kubernetes for AI/ML Workloads

A self-contained ~3-hour lecture + hands-on session: minikube Setup • CPU-Only Distributed Training • Single-Node & Multi-Node Scaling • Scalability Testing

Agenda — ~3 Hours

Time	Block	Format
0:00 – 0:15	From one container to a fleet; core K8s objects	Lecture
0:15 – 0:35	Jobs vs. Deployments; Infrastructure as Code	Lecture
0:35 – 0:55	minikube setup: single-node, then multi-node	Lecture + setup
0:55 – 1:05	Break	—
1:05 – 1:25	The workload: CPU-only distributed hyperparameter search	Lecture
1:25 – 1:45	Indexed Jobs: completions, parallelism, JOB_COMPLETION_INDEX	Lecture + code
1:45 – 2:15	Hands-on: single-node, multi-pod run	Lab
2:15 – 2:45	Hands-on: scale to 3 nodes, multi-node run	Lab
2:45 – 3:00	Scalability testing: does more parallelism actually help?	Lab + discussion

Before You Start This Session

- Docker Desktop or Docker Engine, installed and working (from Lecture 1) — minikube uses it as its driver.
- minikube and kubectl installed — full install commands are on the next slide.
- No GPU required anywhere in this lecture. Every workload here is CPU-only, chosen specifically so it runs on any student laptop.
- At least 4 CPU cores and 4GB RAM free for the single-node demo; the multi-node demo adds 2 more lightweight nodes on top of the same machine (minikube nodes are containers, not separate physical machines).
- This deck is self-contained: every concept is followed by a runnable code snippet or command, and the hands-on exercises (amber slides) reference the companion Jupyter notebook.

00

From One Container to a Fleet

What this section covers

- What Docker answers, and what it doesn't
- Core Kubernetes objects
- Jobs vs. Deployments
- Infrastructure as Code

Docker vs. Kubernetes: Different Questions

Docker Answers

- “How do I package one reproducible unit of work?”
- One image, one container, one machine.

Kubernetes Answers

- “I have dozens of these containers and several machines — who decides what runs where?”
- Schedules containers onto a cluster, restarts failures, exposes services to the network.

The Five Objects You Need First

Object	What It Is	AI/ML Relevance
Pod	Smallest deployable unit — one or more tightly-coupled containers	One training or inference container usually = one Pod
Deployment	Manages identical Pod replicas; handles rolling updates	Scaling an inference service horizontally (Lecture 2b)
Job	Runs Pods to COMPLETION, tracks success/failure, can retry	Today's hands-on: a distributed hyperparameter search
Service	Stable network endpoint for a changing set of Pods	Exposing an inference endpoint at a fixed address
Node	A physical/virtual machine that runs Pods	One minikube node per “machine” in today's cluster

Run-Once vs. Run-Forever

- A TRAINING run is naturally a Kubernetes Job: run to completion, potentially retried on failure, then done. Today's entire hands-on is Jobs.
- An INFERENCE service is naturally a Deployment: run forever, serving traffic, scaled up/down with demand. That's Lecture 2b.
- Using the wrong object type is a common early mistake: a training script deployed as a Deployment gets endlessly restarted the moment it successfully finishes and exits.

Cluster Configuration as a Version-Controlled Artifact

- Every Kubernetes object in this lecture is defined in a DECLARATIVE YAML manifest, not created via ad hoc commands.
- You describe the DESIRED end state (“12 hyperparameter combinations, 4 running at once”); Kubernetes continuously reconciles reality toward that state.
- Manifests live in Git, exactly like code and DVC pointers from Module 1 — a cluster's entire configuration history is auditable.

01

Setting Up minikube

What this section covers

- Installing minikube and kubectl
- Starting a single-node cluster
- Scaling to a 3-node cluster

1.1 INSTALLATION

minikube & kubectl — All Three Platforms

```
# macOS
brew install minikube kubectl

# Linux
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install kubectl /usr/local/bin/kubectl

# Windows (PowerShell, as Administrator)
choco install minikube kubernetes-cli

# Verify on any platform:
minikube version && kubectl version --client
```

minikube needs a driver. Docker Desktop (from Lecture 1) works on all three platforms: minikube config set driver docker

4 CPUs, One Node, to Start

```
$ minikube start --cpus 4 --memory 4096 --driver=docker
```

```
$ kubectl cluster-info
```

```
$ kubectl get nodes -o wide
```

```
# Expected: one node, STATUS = Ready
```

We start with ONE node to clearly show multi-POD parallelism first, before adding more nodes and showing multi-NODE parallelism.

1.3 SCALE TO A 3-NODE CLUSTER

Adding Nodes to a Running Cluster

```
$ minikube node add  
$ minikube node add  
  
$ kubectl get nodes -o wide  
# Expected: 3 nodes, all STATUS = Ready  
  
# Alternative: start fresh with all 3 nodes at once  
$ minikube start --nodes 3 --cpus 2 --memory 4096 --driver=docker
```

minikube nodes are containers on your one machine, not separate physical hardware — but Kubernetes schedules across them exactly as it would across a real multi-machine cluster.

02

The Workload: Distributed Hyperparameter Search

What this section covers

- Why this workload, and why CPU-only
- The dataset and the 12-point grid
- Embarrassingly parallel, by design

Embarrassingly Parallel, No GPU Required

- Training the same model many times with different settings is EMBARRASSINGLY PARALLEL: each run is fully independent, needs no communication with the others.
- This is exactly the kind of workload Kubernetes Jobs are built for — and it requires no GPU, no multi-node networking libraries, nothing beyond plain scikit-learn.
- Each run finishes in under 3 seconds, so the whole exercise runs comfortably on a laptop — you'll see real parallel speedup within minutes, not hours.

2.2 THE DATASET

A Small, Deterministic Synthetic Classification Task

```
from sklearn.datasets import make_classification

X, y = make_classification(
    n_samples=6000, n_features=20, n_informative=12,
    n_redundant=4, n_classes=2, class_sep=1.1,
    random_state=42, # deterministic -- every pod sees identical data
)

# Saved as ml_job_dataset.csv, baked directly into the worker's
# container image -- no shared/networked storage needed anywhere
```

6,000 rows, 20 features, ~2MB as CSV — small enough to bake directly into a Docker image, which sidesteps minikube's node-local storage limitation entirely.

12 Hyperparameter Combinations

```
import itertools

N_ESTIMATORS_GRID = [50, 100, 150, 200]
MAX_DEPTH_GRID = [4, 8, 12]
GRID = list(itertools.product(N_ESTIMATORS_GRID, MAX_DEPTH_GRID))
# -> 12 combinations, e.g. (50, 4), (50, 8), ..., (200, 12)

# Each combination trains a RandomForestClassifier with n_jobs=1
# -- DELIBERATE: one pod should use ~1 CPU core, so the parallelism
# story is about KUBERNETES scheduling pods, not sklearn's own
# internal multithreading.
```

03

Indexed Jobs

What this section covers

- completions, parallelism, and completionMode
- JOB_COMPLETION_INDEX: how each pod knows its assignment

3.1 THREE FIELDS THAT CONTROL EVERYTHING

Job Spec Vocabulary

Field	Meaning
completions	Total number of pods that must succeed — our fixed grid size (12)
parallelism	How many pods run CONCURRENTLY at once — what we vary to test scaling
completionMode: Indexed	Kubernetes assigns each pod a unique index (0..completions-1) via JOB_COMPLETION_INDEX

Reading JOB_COMPLETION_INDEX

```
import os

completion_index = int(os.environ.get("JOB_COMPLETION_INDEX", "0"))
n_estimators, max_depth = GRID[completion_index % len(GRID)]

# Pod 0 trains (50, 4)
# Pod 1 trains (50, 8)
# ...
# Pod 11 trains (200, 12)

# Kubernetes sets this automatically -- nothing in the Job manifest
# needs to enumerate the 12 combinations by hand.
```

This is the entire mechanism that turns 12 independent Python scripts into one coordinated Kubernetes Job.

3.3 THE FULL WORKER SCRIPT

train_worker.py — Complete Logic

```
def main():
    idx = int(os.environ.get("JOB_COMPLETION_INDEX", "0"))
    n_estimators, max_depth = GRID[idx % len(GRID)]
    pod_name = os.environ.get("POD_NAME", socket.gethostname())
    node_name = os.environ.get("NODE_NAME", "unknown")

    df = pd.read_csv(os.environ.get("DATA_PATH"))
    X_train, X_test, y_train, y_test = train_test_split(...)

    model = RandomForestClassifier(n_estimators=n_estimators,
                                   max_depth=max_depth, random_state=42, n_jobs=1)
    model.fit(X_train, y_train)

    result = {"completion_index": idx, "n_estimators": n_estimators,
              "max_depth": max_depth, "accuracy": ..., "pod_name": pod_name,
              "node_name": node_name}
    print("RESULT_JSON:" + json.dumps(result)) # <- parsed later
```

POD_NAME and NODE_NAME come from the Kubernetes Downward API (next slide) — this is what lets us prove, after the fact, which node ran which piece of work.

The Kubernetes Downward API

```
env:  
- name: POD_NAME  
  valueFrom:  
    fieldRef:  
      fieldPath: metadata.name  
- name: NODE_NAME  
  valueFrom:  
    fieldRef:  
      fieldPath: spec.nodeName
```

The Downward API exposes a Pod's own metadata to itself as environment variables — no separate API call needed.

04

Hands-on: Single-Node, Multi-Pod

What this section covers

- Building and loading the worker image
- Applying the Job and watching pods run concurrently

4.1 BUILD THE WORKER IMAGE

Directly Into minikube — No Registry Needed

```
# Multi-stage Dockerfile (Lecture 1 pattern) -- bakes in the dataset
$ minikube image build -t hp-search-worker:latest -f Dockerfile.job .

$ minikube image ls | grep hp-search-worker
```

Building directly inside minikube's image store means the cluster can use the image immediately — no push/pull to an external registry.

4.2 THE JOB MANIFEST

job-single-node.yaml

```
apiVersion: batch/v1
kind: Job
metadata:
  name: hp-search-job
spec:
  completions: 12
  parallelism: 4      # up to 4 pods running AT ONCE
  completionMode: Indexed
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: worker
        image: hp-search-worker:latest
        env: [DATA_PATH, POD_NAME, NODE_NAME] # (Downward API, prior slide)
        resources:
          requests: {cpu: "900m", memory: "256Mi"}
          limits:  {cpu: "1000m", memory: "512Mi"}
```

cpu: "900m" requests roughly one full core per pod — with 4 CPUs available and parallelism: 4, all four run truly concurrently.

4.3 APPLY AND WATCH

See Up to 4 Pods Running Simultaneously

```
$ kubectl apply -f job-single-node.yaml
```

```
$ kubectl get pods -o wide -w
```

NAME	READY	STATUS	NODE	AGE
hp-search-job-0-x7f2q	1/1	Running	minikube	2s
hp-search-job-1-k9m3z	1/1	Running	minikube	2s
hp-search-job-2-p4n8w	1/1	Running	minikube	2s
hp-search-job-3-q2j5r	1/1	Running	minikube	2s

... pods 4-11 queue and run as earlier ones finish ...

Watch the STATUS column: up to 4 Running at once, all on the SAME node (only one exists so far) — this is multi-pod, multi-CPU parallelism.

05

Collecting Results

What this section covers

- Why NOT a shared volume
- Pulling results back via the Kubernetes API

5.1 WHY WE DON'T USE A SHARED VOLUME

minikube's Storage Is Node-Local by Default

- The obvious approach — have every pod write results to a shared PersistentVolume — breaks in the multi-node exercise: minikube's default storage provisioner binds a volume to ONE node.
- A pod scheduled on a DIFFERENT node can't reliably read/write that same volume.
- Instead: each pod prints its result to stdout, and we pull it back via the Kubernetes API — which works identically regardless of which node a pod landed on.

Reading Results from Pod Logs via the Kubernetes API

```
from kubernetes import client, config
import re, json

config.load_kube_config()
v1 = client.CoreV1Api()

pods = v1.list_namespaced_pod(
    namespace="default", label_selector="job-name=hp-search-job")

for pod in pods.items:
    logs = v1.read_namespaced_pod_log(pod.metadata.name, "default")
    match = re.search(r"RESULT_JSON:(\{.*\})", logs)
    result = json.loads(match.group(1))
    # result includes: accuracy, n_estimators, max_depth,
    #                  pod_name, node_name
```

The job-name label is applied automatically by Kubernetes to every pod a Job creates — that's how we find all 12 pods.

06

Hands-on: Scaling to 3 Nodes

What this section covers

- Loading the image onto every node
- Spreading pods with topologySpreadConstraints
- Confirming work landed on multiple nodes

6.1 LOAD THE IMAGE ONTO EVERY NODE

A Step That's Easy to Miss

```
$ minikube image load hp-search-worker:latest  
  
# Verify on EACH node -- an image built/loaded on one node is  
# NOT automatically present on the others:  
$ minikube ssh --node minikube docker images  
$ minikube ssh --node minikube-m02 docker images  
$ minikube ssh --node minikube-m03 docker images
```

If a pod lands on a node without the image, you'll see ImagePullBackOff — the fix is always this same minikube image load command.

job-multi-node.yaml — What Changed

```
spec:
  completions: 12
  parallelism: 6      # raised: 3 nodes x ~2 CPUs each
  completionMode: Indexed
  template:
    spec:
      topologySpreadConstraints:
        - maxSkew: 1
          topologyKey: kubernetes.io/hostname
          whenUnsatisfiable: ScheduleAnyway
          labelSelector:
            matchLabels: {app: hp-search}
      # ... containers section unchanged from job-single-node.yaml ...
```

topologySpreadConstraints tells the scheduler to actively balance pods across nodes, rather than incidentally packing them onto whichever node it checks first.

Watch the NODE Column This Time

```
$ kubectl apply -f job-multi-node.yaml
```

```
$ kubectl get pods -o wide -w
```

NAME	NODE	STATUS
hp-search-job-multinode-0-x7f2q	minikube	Running
hp-search-job-multinode-1-k9m3z	minikube-m02	Running
hp-search-job-multinode-2-p4n8w	minikube-m03	Running
hp-search-job-multinode-3-q2j5r	minikube	Running

... pods spread across all 3 nodes as they run ...

This is the direct visual proof that Kubernetes is scheduling work across MULTIPLE MACHINES, not just multiple processes on one.

07

Scalability Testing

What this section covers

- Does more parallelism actually help?
- Sweeping parallelism and measuring wall-clock time

Sweep Parallelism, Fixed Total Work

```
def run_job_at_parallelism(parallelism, completions, ...):
    job_manifest["spec"]["parallelism"] = parallelism
    job_manifest["spec"]["completions"] = completions # FIXED at 12

    start = time.time()
    batch_v1.create_namespaced_job(namespace, job_manifest)
    while True:
        status = batch_v1.read_namespaced_job_status(...).status
        if status.succeeded >= completions:
            break
        time.sleep(2)
    return time.time() - start

# Run this at parallelism = 1, 2, 4 (and 6 on the 3-node cluster)
```

Total work (completions=12) stays constant across every run — only parallelism changes — so any speedup we measure is attributable to Kubernetes scheduling more pods concurrently.

Wall-Clock Time Drops as Parallelism Rises

parallelism=1 -> ~14.2s (all 12 jobs run one after another)

parallelism=2 -> ~7.6s (~1.9x speedup)

parallelism=4 -> ~4.3s (~3.3x speedup)

Sub-linear speedup is EXPECTED and worth discussing:

- uneven work sizes (some hyperparameter combos train faster

than others) mean the LAST pod to finish sets the wall-clock time

- scheduling/startup overhead grows with more concurrent pods

Full methodology + a live matplotlib chart in the companion notebook — run it yourself and compare against these reference numbers.

08

GPU Scheduling (Conceptual)

What this section covers

- The same request/limit pattern, for when GPUs ARE available
- Why today's hands-on stayed CPU-only

8.1 GPU RESOURCES FOLLOW THE SAME PATTERN

For Reference — Not Used in Today's Hands-On

```
resources:  
requests:  
  nvidia.com/gpu: 1  
limits:  
  nvidia.com/gpu: 1 # GPUs cannot be overcommitted -- request must equal limit
```

```
# A device plugin (typically the NVIDIA device plugin) advertises  
# each node's GPU count to the scheduler, letting Kubernetes treat  
# GPUs like CPU/memory -- exactly the pattern used for our  
# cpu: "900m" requests today, just with a different resource name.
```

Unlike CPU, GPU requests and limits MUST be equal. Everything else — scheduling, resource accounting — works identically to what you just ran with cpu:.

09

Case Studies

What this section covers

- A fault-tolerant training job that survives node failures
- Cost-optimization of GPU clusters using spot instances

Surviving Node Failures Without Losing Progress

1

1. The Naive Fix

`backoffLimit` lets a whole Job retry on failure — but restarts training FROM SCRATCH.

2

2. Add Checkpointing

The training container periodically logs a checkpoint (recall Module 1's MLflow logging).

3

3. Use Durable Storage

Checkpoints MUST go to a `PersistentVolume` or object storage — exactly why today's results used the Kubernetes API instead.

4

4. Resume, Don't Restart

A retried Pod resumes from the last checkpoint instead of starting over.

Cost-Optimization with Spot Instances

- Recall Module 2's Ray discussion of spot/preemptible instances — this is how it's implemented at the Kubernetes layer.
- A dedicated spot-instance NODE POOL is tainted, so only workloads that explicitly tolerate interruption are scheduled there.
- Pod PRIORITY CLASSES let critical workloads preempt lower-priority ones if a spot node disappears unexpectedly.
- Discussion: today's hyperparameter search Job is a PERFECT spot-instance candidate — why? (Each pod is independent, cheap to retry, and short-lived.)

Distributed Hyperparameter Search: Single-Node, Then Multi-Node

1. Open the companion notebook: `Lecture2a_Kubernetes_Scaling_Exercise.ipynb`.
2. Install minikube/kubectl if needed; start a single-node, 4-CPU cluster.
3. Write the dataset generator and training worker script; sanity-check the worker locally.
4. Build the worker image with minikube image build; apply `job-single-node.yaml`; watch pods run concurrently.
5. Collect and inspect the 12 results via the Kubernetes API — confirm all ran on the one node.
6. Scale to 3 nodes (minikube node add x2); load the image onto every node; apply `job-multi-node.yaml`.
7. Confirm results now show ≥ 2 distinct `node_name` values; run the scalability sweep and inspect the plot.

Deliverable: Pod/node evidence from both runs, the 12-row results table from each, and the parallelism-vs-time scalability plot.

Kubernetes Cheat Sheet

Task	Command / Code
Install & verify	minikube version / kubectl version --client
Start a cluster	minikube start --cpus 4 --memory 4096 --driver=docker
Add nodes to reach 3	minikube node add (run twice)
Build an image into minikube	minikube image build -t <name> -f <Dockerfile> .
Load an image onto every node	minikube image load <name>
Apply a manifest	kubectl apply -f <file>.yaml
Watch pods (with NODE column)	kubectl get pods -o wide -w
Read a pod's logs	kubectl logs <pod-name>
Indexed Job fields	completions / parallelism / completionMode: Indexed

KEY TAKEAWAY

Docker packages one unit. Kubernetes decides where hundreds of them run:

Indexed Jobs + parallelism + topology spreading = measurable, provable scaling across CPUs AND nodes

Every number in today's scalability plot came from your OWN cluster — this is what “scales horizontally” looks like as evidence, not just as a claim.

Next: Lecture 2, Part B — Scaling a Model-Serving Deployment with KServe.